Optimization in Deep Neural Networks

Jiwoo Kim

Seoul National University

March 26, 2025

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

Optimization Problem Setting

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(\phi(\mathbf{x}_i^{(0)}; \boldsymbol{\theta}), y_i^{(0)}\right)$$

- $\boldsymbol{\theta} = (\boldsymbol{W}, \boldsymbol{b})$: Parameters (Weight matrices, bias vectors) of DNN ϕ.
- $\{(\mathbf{x}_i^{(0)}, y_i^{(0)})\}_{1 \le i \le n}$: Training dataset.
- $\mathcal{L}(\cdot)$: Loss function measuring discrepancy between truth and prediction.

The Challenge: High-Dimensional Complexity

Deep learning models often involve **millions of parameters**, making the loss landscape highly non-convex and complex.

- **Analytical solution is infeasible:** Cannot simply set $\nabla_{\boldsymbol{\theta}} \mathcal{L} = 0$.
- **Solution:** Employ **Gradient Descent**, an iterative first-order optimization method, to navigate the high-dimensional landscape.

Batch Gradient Descent (BGD)

- Definition: Uses the **entire dataset** to perform a single parameter update.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta_t}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\phi(\boldsymbol{x}_i^{(0)}; \boldsymbol{\theta}_t), y_i^{(0)})$$

  - $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\cdot)$: Gradient of the loss with respect to $\boldsymbol{\theta}$
  - η_t: Learning rate (step size)

- Computational Bottleneck: Extremely slow and memory-intensive for large-scale data.
- Convergence: Guaranteed to reach a global minimum (convex) or a local minimum (non-convex).

Stochastic Gradient Descent (SGD)

- Definition: Performs a parameter update for **each training sample** $(\boldsymbol{x}_i^{(0)}, y_i^{(0)})$.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} \mathcal{L}(\phi(\boldsymbol{x}_i^{(0)}; \boldsymbol{\theta}_t), y_i^{(0)})$$

- High Efficiency: Rapid updates with significantly lower computational latency.
- Stochastic Fluctuation: High variance causes heavy oscillations in the loss, making convergence noisy.

Mini-Batch Gradient Descent

- Definition: Performs an update for **every mini-batch** $\mathcal{B} = \{(\boldsymbol{x}_i^{(0)}, y_i^{(0)})\}_{1 \le i \le m}$ of size $m \ll n$.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta_t}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\phi(\boldsymbol{x}_i^{(0)}; \boldsymbol{\theta}_t), y_i^{(0)}))$$

  - 1 Epoch $= \frac{n}{m}$ updates.

- Stable Convergence: Significantly reduces update variance compared to SGD, leading to a smoother path.
- Hardware Efficiency: Leverages highly optimized matrix operations (vectorization) on GPUs.
- Batch Size (m): Typically ranges from 32 to 512, balancing computational speed and convergence stability.

Comparison between BGD and SGD (Keskar et al., 2016)

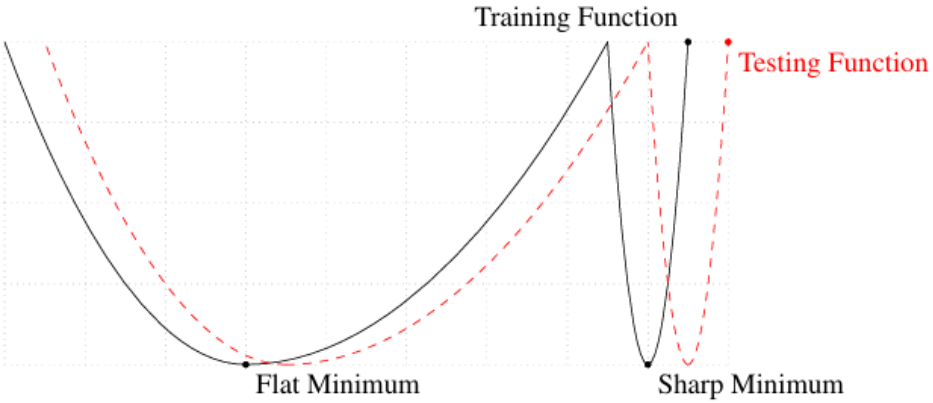


Figure 1: A Conceptual Sketch of Flat and Sharp Minimum. The Y-axis indicates value of the loss function and the X-axis the variables (parameters)

- Training Function: Represents empirical risk.
- Testing Function: Represents expected risk.
- Sharp Mininum: Large eigenvalues of Hessian $\nabla^2_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{BGD}}$
  - High Curvature $\rightarrow$ Sensitive to small shifts $\rightarrow$ Poor Generalization.
- Flat Minimum: Small eigenvalues of Hessian $\nabla^2_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{SGD}}$
  - Low Curvature $\rightarrow$ Robust to small shifts $\rightarrow$ Better Generalization.

Beyond Basic Gradient Descent

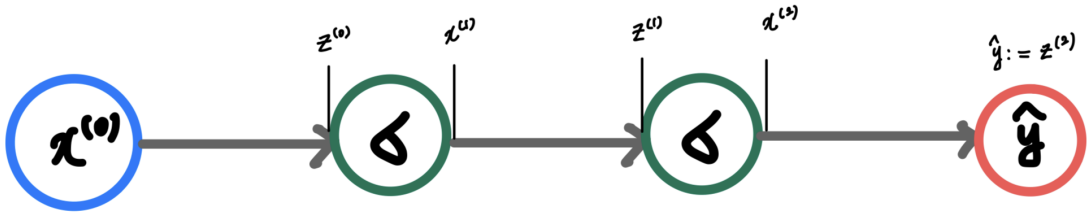
- **Gradient Computation & Backpropagation**
  - Understanding how gradients flow via the chain rule and automatic differentiation.
- **Overcoming Vanishing Gradients**
  - Addressing gradient decay in deep neural networks through new activation function and normalization.
- **Parameter Initialization Strategies**
  - Preventing gradient decay or explosion by setting optimal starting points.
- **Advanced Optimizers**
  - Improving convergence using momentum and adaptive learning rates.

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

Backpropagation

- Idea: Computing loss gradients via **the chain rule**.
- Example



- Forward Pass:

$$x^{(0)} \to z^{(0)} \to x^{(1)} \to z^{(1)} \to x^{(2)} \to \hat{y} \to \mathcal{L}$$

- Backward Pass:

$$\frac{\partial \mathcal{L}}{\partial w^{(0)}} = \underbrace{\underbrace{\underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}}}_{\text{Vector}} \cdot w^{(2)} \cdot \sigma'(z^{(1)}) \cdot w^{(1)} \cdot \sigma'(z^{(0)})}_{\text{Chain of VJPs (Accumulated Gradient)}} \otimes x^{(0)}}_{\text{Outer Product for Weight Update}}$$

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

Vanishing Gradients

- Gradient Calculation: Recursive multiplication of activation derivatives and weight parameters.
- Vanishing Gradients: In DNNs using Sigmoid or Tanh, gradients near the input layer diminish toward zero due to the **repeated multiplication of small activation derivatives**.

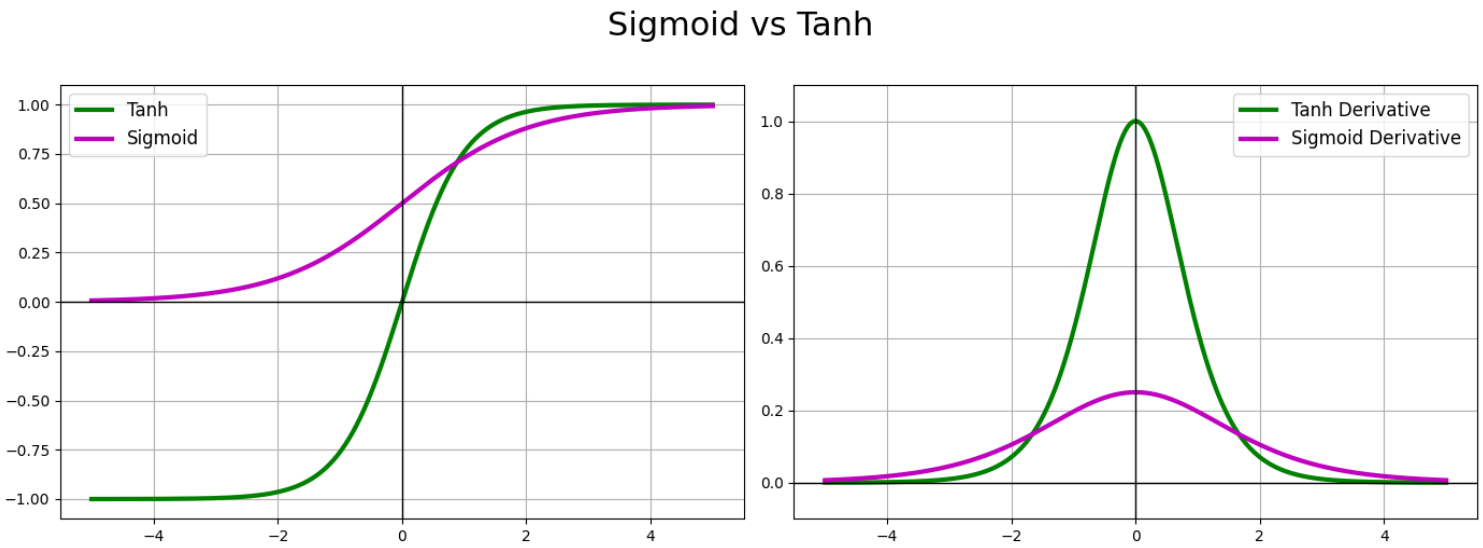


Figure 2: Activation Functions: Sigmoid & Tanh

Rectified Linear Unit (Glorot et al., 2011)

- Rectified Linear Unit(ReLU): $\sigma_{ReLU}(x) = \max(0, x)$
- Constant Gradient ($x > 0$): The gradient is **exactly 1** for all positive inputs.
- Key Advantage: Enables effective training across all layers, including those near the input, by preventing gradient decay.

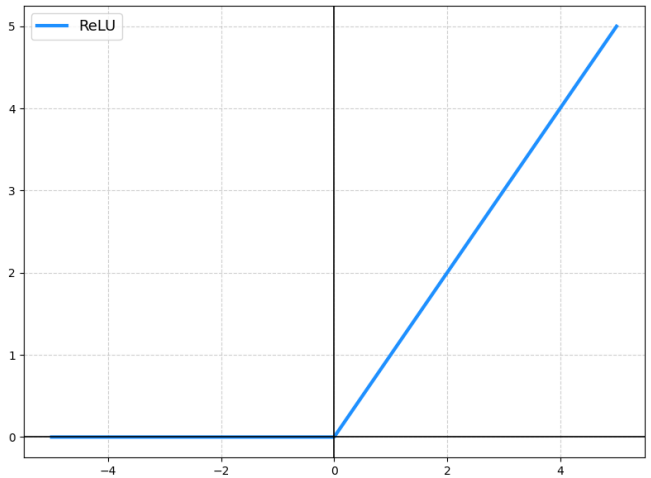


Figure 3: Activation Function: ReLU

Beyond ReLU: Dealing with Internal Covariate Shift

- Limit of ReLU: While it solves the saturation problem, it doesn't stabilize the distribution of pre-activations across a mini-batch $\{z_i^{(l)}\}_{1 \le i \le m}$.
- Internal Covariate Shift (ICS):
  - Change in the distribution of network activations due to the update of previous layers' parameters.

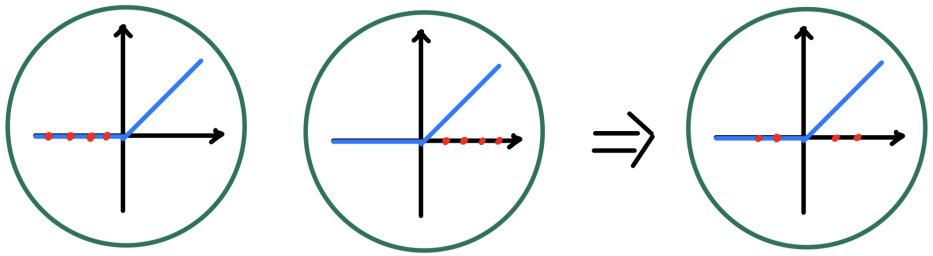


Figure 4: Impact of ICS on ReLU: Distribution Shift

Batch Normalization (Ioffe and Szegedy, 2015)

- BN Transform: Applied to each dimension k independently for a mini-batch $\mathcal{B}_k = \{z_{i,k}^{(l)}\}_{1 \le i \le m}$.

BN Transform Steps

1. **Mini-Batch Mean:** $\mu_k = \frac{1}{m} \sum_{i=1}^{m} z_{i,k}$
2. **Mini-Batch Variance:** $\sigma_k^2 = \frac{1}{m} \sum_{i=1}^{m} (z_{i,k} - \mu_k)^2$
3. **Normalization:** $\hat{z}_{i,k} = \frac{z_{i,k} - \mu_k}{\sqrt{\sigma_k^2 + \epsilon}}$ (where ϵ is for numerical stability)
4. **Scale and Shift:** $y_{i,k} = \gamma_k \hat{z}_{i,k} + \beta_k$ for $i = 1, \ldots, m$

- **Learnable Parameters** γ, β: Allowing the model to restore expressive power (Identity Transform) by learning optimal distributions beyond $N(0, 1)$.

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

Parameter Initialization: Fundamental Concepts

- Weights ($\boldsymbol{W}$): Randomly initialized (e.g., Normal) with **mean zero**.
- Bias ($\boldsymbol{b}$): Typically initialized to **zero** to prevent unwanted shifts in initial distributions.

- Structural Metrics:
  - Fan-in ($d_{in}^{(l)}$): Number of input nodes of a specific layer l.
  - Fan-out ($d_{out}^{(l)}$): Number of output nodes of a specific layer l.

- **Core Objective: Variance Preservation**
  - Keeping the signal intensity stable across the entire network
  - $Var(\text{Input}) \approx Var(\text{Output})$

- Dual Perspectives:
  - Forward Pass: Prevents signal vanishing ($\rightarrow 0$) or exploding ($\rightarrow \infty$).
  - Backward Pass: Ensures stable gradient flow ($\nabla_{\boldsymbol{\theta}} \mathcal{L}$) for effective learning.

Xavier Initialization (Glorot and Bengio, 2010)

- Target: Optimized for **symmetric** activations (Sigmoid, Tanh).
- Assumptions:
  - Inputs: Zero-centered ($E[\boldsymbol{x}^{(0)}] = 0$) and normalized.
  - Weights: i.i.d. with $E[\boldsymbol{w}^{(l)}] = 0$.
  - Linearity: $\sigma'(0) \approx 1$ (Near-linear at origin).
  - Independence: $\boldsymbol{W}, \boldsymbol{x}$ are mutually independent.

Forward Pass

For each output node $j \in \{1, \ldots, d_{out}^{(l)}\}$:

$$
\begin{aligned}
z_j^{(l)} &= \sum_{i=1}^{d_{in}^{(l)}} w_{ji}^{(l)} x_i^{(l)} + b_j^{(l)} \\
Var[z_j^{(l)}] &= d_{in}^{(l)} Var[w^{(l)}] Var[x^{(l)}] \quad (\because E[w] = E[x] = 0, b = 0) \\
&\approx d_{in}^{(l)} Var[w^{(l)}] Var[z^{(l-1)}] \quad (\because \sigma(z) \approx z) \\
&\implies Var[w^{(l)}] = 1/d_{in}^{(l)}
\end{aligned}
$$

Xavier Initialization

- Assumptions:
  - Gradients: Zero-centered ($E[\nabla \boldsymbol{z}^{(l)}] = 0$) for all layers.
  - Weights: i.i.d. with $E[\boldsymbol{w}^{(l)}] = 0$.
  - Linearity: $\sigma'(0) \approx 1 \implies \nabla \boldsymbol{z}^{(l)} \approx \nabla \boldsymbol{x}^{(l+1)}$.
  - Independence: $\boldsymbol{W}$ and $\nabla \boldsymbol{z}$ are mutually independent.

Backward Pass

For the gradient of each input node $i \in \{1, \ldots, d_{in}^{(l)}\}$:

$$\nabla x_i^{(l)} = \sum_{j=1}^{d_{out}^{(l)}} w_{ji}^{(l)} \nabla z_j^{(l)} \quad \text{(Backpropagation through } \boldsymbol{W}^T\text{)}$$

$$\begin{aligned} Var[\nabla x_i^{(l)}] &= d_{out}^{(l)} Var[w^{(l)}] Var[\nabla z^{(l)}] \quad (\because E[w] = E[\nabla z] = 0) \\ &\approx d_{out}^{(l)} Var[w^{(l)}] Var[\nabla x^{(l+1)}] \quad (\because \sigma'(z) \approx 1) \\ &\implies Var[w^{(l)}] = 1/d_{out}^{(l)} \end{aligned}$$

Xavier Initialization

Final Xavier Strategy (The Compromise)

The optimal parameter variance balances both signal and gradient flow:

$$Var(w^{(l)}) = \frac{2}{d_{in}^{(l)} + d_{out}^{(l)}} \quad \Rightarrow \quad \boldsymbol{W} \sim \mathcal{N}\left(0, \frac{2}{d_{in}^{(l)} + d_{out}^{(l)}}\right)$$

He Initialization (He et al., 2015)

- Target: Optimized for **non-symmetric** rectifiers (ReLU, PReLU).
- Assumptions:
  - Symmetry: Pre-activation $\boldsymbol{z}^{(l-1)}$ is symmetric around zero ($E[\boldsymbol{z}^{(l-1)}] = 0$).
  - ReLU Effect: $\boldsymbol{x}^{(l)} = \max(0, \boldsymbol{z}^{(l-1)})$, implying $E[\boldsymbol{x}^{(l)}] \neq 0$.
  - Weights: i.i.d. with $E[\boldsymbol{w}^{(l)}] = 0$.
  - Independence: $\boldsymbol{w}^{(l)}$ and $\boldsymbol{x}^{(l)}$ are mutually independent.

Forward Pass: Addressing the "Half-Zero" Problem

To preserve variance $Var[z_j^{(l)}] \approx Var[z_i^{(l-1)}]$ under ReLU:

$$
\begin{aligned}
Var[z_j^{(l)}] &= d_{in}^{(l)} Var[w^{(l)}] E[(x^{(l)})^2] \quad (\because E[w] = 0, b = 0) \\
&= d_{in}^{(l)} Var[w^{(l)}] \left(\frac{1}{2} Var[z^{(l-1)}] \right) \quad (\because \text{ReLU cuts half of } z) \\
&\implies Var[w^{(l)}] = \frac{2}{d_{in}^{(l)}} \\
&\implies \boldsymbol{W} \sim \mathcal{N}\left(0, 2/d_{in}^{(l)}\right)
\end{aligned}
$$

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

Momemtum Optimizer (Qian, 1999)

- The Problem: Standard SGD in ravines
  - Ravine: Areas where the surface curves much more steeply in one dimension than in another.
  - Issue: High-frequency oscillations lead to inefficient and slow convergence.

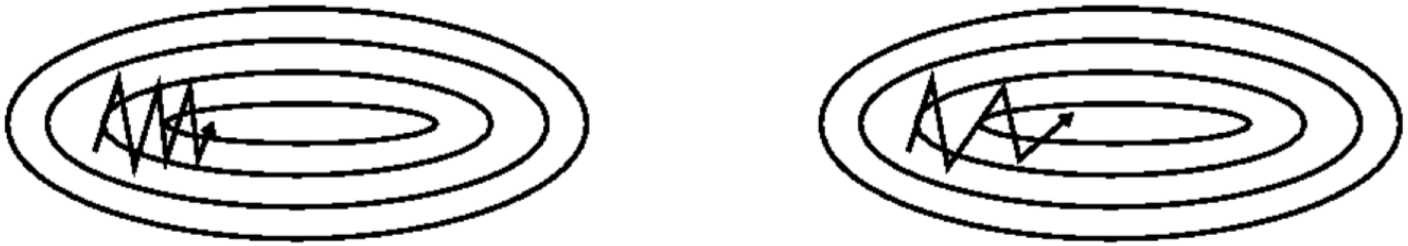


Figure 5: Left: SGD w/o Momentum Right: SGD w/ Momemtum

Momentum Optimizer

- Idea: Accumulates the gradient by adding a fraction (γ) of the past velocity to the current gradient.

$$\mathbf{m}_t = \gamma \mathbf{m}_{t-1} + \eta_t \boldsymbol{g}_t$$
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{m}_t$$

- Components:
  - γ (Momentum term): Usually set to 0.9.
  - $\boldsymbol{g}_t$: gradient at time t ($\boldsymbol{g}_t := \frac{1}{m}\sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\phi(\boldsymbol{x}_i^{(0)}; \boldsymbol{\theta}_t), y_i^{(0)})$).
  - $\mathbf{m}_t$: Exponential moving average of gradients.

- Effect: Subtracting the accumulated velocity $\mathbf{m}_t$ effectively dampens oscillations and speeds up learning.

Root Mean Sqaure Propagation: RMSProp Otimizer

- Idea: **Scale the learning rate** for each parameter based on the root mean square of recent gradients.

$$\boldsymbol{v}_t = \beta \boldsymbol{v}_{t-1} + (1-\beta)\boldsymbol{g}_t^2$$
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta_t}{\sqrt{\boldsymbol{v}_t + \epsilon}}\boldsymbol{g}_t$$

- Components:
  - β (Decay rate): Usually set to 0.9.
  - ϵ (Smoothing term): 10^{-8}.
  - $\mathbf{v}_t$: Exponential moving average of squared gradients.

- Effect: Accelerates progress in flat plateaus (small g_t) while stabilizing updates in steep ravines (large g_t).

Adaptive Moment Estimation: Adam Optimizer (Kingma and Ba, 2014)

- The "Cold Start" Problem:
  - Moments are initialized to zero: $\boldsymbol{m}_0 = \mathbf{0}, \boldsymbol{v}_0 = \mathbf{0}$.
  - During early iterations, $\boldsymbol{m}_t$ and $\boldsymbol{v}_t$ are heavily **biased toward zero**.

Key Characteristic: Self-Vanishing Correction

- As $t \to \infty$, the term β^t decays to 0.
- The correction factor $(1 - \beta^t) \to 1$, meaning the bias correction **automatically fades out** as the training stabilizes.

Adaptive Moment Estimation: Adam Optimizer (Kingma and Ba, 2014)

- Idea: Combines the advantages of Momentum (1st Moment) and RMSProp (2nd Moment) to achieve stable and adaptive learning.

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t \quad \text{(1st Moment: Momentum)}$$
$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2 \quad \text{(2nd Moment: RMSProp)}$$

- Hyperparameters: $\beta_1 = 0.9, \beta_2 = 0.999$: Decay rates for moments.
- The Cold Start Problem:
  - Moments are initialized to zero: $\boldsymbol{m}_0 = \mathbf{0}, \boldsymbol{v}_0 = \mathbf{0}$.
  - During early iterations, $\boldsymbol{m}_t$ and $\boldsymbol{v}_t$ are heavily **biased toward zero**.
- **Bias Correction**

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \quad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}$$

- Update Rule:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}\hat{\boldsymbol{m}}_t$$

Table of Contents

1. Gradient Descent
2. Gradient Computaion & Backpropagation
3. Overcoming Vanishing Gradients
4. Parameter Initialization Strategies
5. Advanced Optimizers
6. Reference

References I

Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings.

Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323. JMLR Workshop and Conference Proceedings.

He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034.

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr.

References II

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2016). On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*.

Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural networks*, 12(1):145–151.

Thank You